StudyForge AI

Full Backend Explanation

From Python basics to every important route, database decision and fallback

Backend/app/main.py and backend/app/study.py explained

1 Backend Purpose and Mental Model

The backend is the trusted brain of StudyForge. The frontend can show forms and buttons, but the backend decides whether a request is allowed, reads the PDF, stores information, searches study text and communicates with AI providers. It runs as a FastAPI server on port 8000.

A beginner should remember this sentence: the backend receives an HTTP request, validates it, performs trusted work, then returns a JSON response. In StudyForge, trusted work includes password hashing, document ownership checks, SQLite access, PDF text extraction and AI communication.

Input	Backend work	Output
Sign-up form	Validate, hash password, insert user	Token and public user
PDF upload	Validate, extract, chunk, store	Document ID and page count
Chat question	Authorize, retrieve, generate or fallback	Answer and sources
Quiz request	Authorize, create distinct items, store answers	Safe question list

Viva question: Why is backend separate from frontend? Answer: Because private data, secrets and permission checks must not be trusted to browser code.

2 Backend Folder Map

```
backend/
  app/main.py        Core FastAPI app, database, auth, upload, chat, quiz
  app/study.py       Library, subjects, tasks, revision and conversation router
  app/config.py      Small environment settings object
  app/services/      Service boundaries and helper prototypes
  app/routes/        Route package placeholder
  tests/             pytest test modules
  requirements.txt   Python dependencies
  .env.example       Example environment variables
```

main.py is the main integration file. It contains the real running APIs. study.py is a feature router created by create_router and added at the end of main.py. This separation keeps library, planner, revision and saved conversation endpoints out of the already large core file.

File	When to open it
main.py	Accounts, PDF, RAG, AI, flashcards, quiz, progress.
study.py	Library, subject assignment, downloads, tasks, topics, conversations.
requirements.txt	Explain backend packages.
tests	Explain verification.
.env.example	Explain configuration without exposing secrets.

Viva question: What is a router? Answer: A collection of related API routes that can be attached to the main FastAPI application.

3 Startup Configuration and Dependencies

At startup, main.py imports Python standard library modules, FastAPI, Pydantic, PyMuPDF, scikit-learn and Google GenAI. It finds a data directory, upload directory and SQLite database path. It creates the upload directory if it does not exist, then calls initialize_database.

Package	Use in StudyForge
fastapi	Create routes, dependencies and HTTP exceptions.
uvicorn	Run FastAPI server.
python-multipart	Accept multipart PDF upload.
PyMuPDF fitz	Open PDFs and extract page text.
scikit-learn	TF IDF and cosine similarity retrieval.
google-genai	Gemini client and generation.
python-dotenv	Load local environment values.
httpx	Groq streaming HTTP calls.
pytest	Run automated checks.

Configuration comes from environment variables. GEMINI_API_KEY is never written in source code. A developer sets it locally in .env or environment. This is important because Git history can be public.

Viva question: Why use environment variables? Answer: They keep changeable and secret settings outside source code.

4 Database Startup and Migration

initialize_database creates the tables needed by the project. CREATE TABLE IF NOT EXISTS makes the code safe to run repeatedly. The function also checks existing table columns with PRAGMA table_info and adds newer columns where required. This is a lightweight migration technique.

```
CREATE TABLE IF NOT EXISTS users (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  ...
)
```

The project does not use an ORM. It uses sqlite3 directly with parameterized SQL. This keeps the code understandable for a college project. db_connection opens SQLite and sets row_factory to sqlite3.Row, so code can use row['email'] rather than numeric index.

Migration action	Why
Create missing table	First run creates initial database.
Check document user_id	Supports older databases.
Add profile columns	Adds branch, semester, college, avatar and gender.
Add task columns	Adds subject, priority and notes.
Create quiz_attempts	Stores score history.

Viva question: What is database migration? Answer: A controlled change to database structure without deleting existing useful data.

5 Data Models and Validation

Pydantic BaseModel classes describe request data. FastAPI automatically reads JSON into these models and validates field types and limits. This means route code receives cleaner input.

Model	Used by	What it validates
ChatMessage	Chat history	role and content
ChatRequest	Chat routes	question, document ID, mode and history
SignupRequest	Signup	name, email and password
LoginRequest	Login	email and password
ProfileRequest	Profile update	preferences plus branch and college
GenerateRequest	Cards and quizzes	document ID and count 1 to 20
QuizSubmission	Quiz submit	answers dictionary
AvatarRequest	Avatar update	avatar data and gender

Field constraints are not decoration. For example, count has a lower and upper limit. This prevents unnecessary provider cost or huge response. Validators in study.py trim a subject name and reject whitespace-only names.

Viva question: What does Pydantic do? Answer: It parses and validates request data before route logic uses it.

6 Password Hashing

The backend must never store raw passwords. password_hash creates a random 16-byte salt and calls hashlib.pbkdf2_hmac with SHA 256 and 120000 iterations. The saved string contains salt and digest separated by a dollar sign.

```
salt = os.urandom(16)
digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 120_000)
return salt.hex() + "$" + digest.hex()
```

At login, password_matches splits the stored value, recomputes a digest using the entered password and same salt, then uses hmac.compare_digest. A salt makes equal passwords produce different stored hashes. Repeated iterations make brute-force attempts slower.

Viva question: Why is a hash not encryption? Answer: Encryption can be reversed using a key. A password hash is designed for one-way verification.

7 Token Authentication

After login, create_token makes a payload containing user ID and expiry time seven days in the future. It signs that payload using HMAC SHA 256 and the server token secret. The result is base64 encoded and returned to the browser.

For later requests, frontend sends Authorization Bearer token. current_user checks that header exists, decodes it, splits user ID, expiry and signature, computes expected signature and rejects bad or expired values. It then loads the user record.

Step	Reason
Create secret	Server needs stable signing secret.
Create payload	Identifies user and expiry.
Sign with HMAC	Stops simple modification of payload.
Send Bearer header	Identifies logged-in request.
Validate in dependency	Every protected route gets verified user.

This is a custom signed session token, not a standard JWT package. It is acceptable to explain it as the current implementation and to say standard sessions or JWT libraries are future production improvements.

Viva question: What does Depends(current_user) do? Answer: FastAPI runs authentication first and passes the verified SQLite user row into the route.

8 Profile and Public User Data

public_user changes a SQLite user row into safe response data. It removes password_hash and converts subjects and avatar from JSON strings back into Python lists or dictionaries. This prevents accidental password-hash exposure.

Profile endpoints update preferences, branch, semester, college and avatar. update_avatar stores the avatar model as JSON and saves gender. Storing flexible avatar properties as JSON is practical because appearance data may grow without requiring a database column for every color or accessory.

Route	Purpose
PUT /api/profile/preferences	Save subjects, goal, explanation style, difficulty and theme.
PUT /api/profile	Save broader student profile.
PUT /api/profile/avatar	Save avatar object and gender.
GET /api/auth/me	Return safe current user profile.

Viva question: Why remove password_hash from response? Answer: A frontend never needs it, and sending it would expose sensitive security information.

9 PDF Upload Route

POST /api/documents/upload receives UploadFile and verified user. It checks content_type equals application/pdf. It reads at most 20 MB plus one byte, rejecting any larger file with 413. It opens the bytes through fitz and rejects encrypted PDFs or documents outside 1 to 200 pages.

For every PDF page, the backend calls page.get_text('text'), trims the result and passes it to split_text. It creates tuples containing document ID, physical page number and chunk text. Then it writes original bytes to upload directory and inserts document plus chunks into SQLite.

```
with fitz.open(stream=content, filetype="pdf") as pdf:
    for page_number, page in enumerate(pdf, 1):
        for text in split_text(page.get_text("text").strip()):
            chunks.append((document_id, page_number, text))
```

The response includes document ID, filename, chunk count, page count and scanned flag. A scanned flag is true when no text chunks are available.

Viva question: Why save both PDF and chunks? Answer: PDF is original source; chunks are fast searchable text with page references.

10 Retrieval and TF IDF

retrieve receives question, optional document ID and limit. It loads chunks from SQLite. If document ID is present, it searches only that document. Otherwise it can search all chunks. It then creates a TF IDF matrix containing all chunk texts plus the question.

The final vector in the matrix is the question. cosine_similarity compares it against earlier chunk vectors. The code sorts scores descending, takes the top limit, removes zero-score rows and returns document ID, page, excerpt and rounded score.

```
matrix = TfidfVectorizer(stop_words="english").fit_transform(texts + [question])
scores = cosine_similarity(matrix[-1], matrix[:-1]).flatten()
best = scores.argsort()[::-1][:limit]
```

TF IDF is lexical retrieval. It works from words. It is fast and local but may miss semantic synonyms. The guide should present embeddings and vector databases as future scope, not present code.

Viva question: What does retrieve return? Answer: Relevant chunk excerpts with source page and similarity score.

11 Gemini Prompt and Client

gemini_payload prepares messages for Gemini. It uses the student question, retrieved context, selected study mode and recent history. The goal is a response that is useful to a BTech student and connected to uploaded material.

gemini_client creates a Google GenAI client only if GEMINI_API_KEY exists. ask_gemini calls it for a non-streaming response. check_gemini_finish checks whether provider stopped normally; incomplete or safety-limited content is treated as unsuccessful instead of silently accepted.

generate_gemini_content has a bounded fallback model strategy for some provider failures. It does not retry endlessly. This matters because uncontrolled retry can make a failed request slow and expensive.

Function	Responsibility
gemini_client	Create configured provider client.
gemini_payload	Build safe context and prompt.
ask_gemini	Get normal answer.
check_gemini_finish	Reject incomplete generation.
generate_gemini_content	Use controlled provider fallback.

Viva question: Why validate provider output? Answer: An AI response can be incomplete or malformed; backend must not trust it automatically.

12 Chat Preparation and Result

prepare_chat is a useful central function. It checks the selected document belongs to current user, obtains retrieved sources, locates original PDF if present and creates arguments for the provider. This avoids repeating the same setup in both normal chat and streaming chat routes.

chat_result packages final answer, sources, document information and grounding details for frontend. chat_failure maps technical provider errors to understandable student-facing messages such as temporary busy, key rejected or request limit reached.

Route	Behaviour
GET /api/chat/status	Tells frontend which chat provider is configured.
POST /api/chat	Returns normal completed JSON answer.
POST /api/chat/stream	Returns newline-delimited streaming events.
prepare_chat	Shared authorization and retrieval.
chat_result	Shared response formatting.

Viva question: Why share prepare_chat between routes? Answer: It prevents normal and streaming chat from applying different ownership or retrieval rules.

13 Streaming Chat Route

chat_stream returns StreamingResponse with media type application/x-ndjson. Its internal events generator first yields status, then repeatedly yields delta text, then yields a done event. If something fails, it yields error event. The frontend can parse every line separately.

```
yield event({"type": "status", "message": "Reading your PDF…"})
for text in chunks:
    answer += text
    yield event({"type": "delta", "text": text})
yield event({"type": "done", **result})
```

The route closes the chunk iterator in finally if it has a close method. This is good cleanup. The route also checks an empty provider response and translates it to a controlled failure.

Viva question: Why stream instead of one normal response? Answer: Student sees progress and text immediately instead of waiting for a full long AI answer.

14 Flashcard Generation

generate_document_items is shared for cards and quizzes. It asks Gemini for more candidates than required, requests broad coverage of named concepts and asks it not to repeat concepts with different wording. Every candidate needs a source page.

For flashcards, required output includes question, focus label, answer and source page. If AI cards are unavailable, make_flashcards creates basic cards from readable stored chunk sentences. The fallback uses varied question stems and distinct-item filtering.

POST /api/flashcards/generate first checks document ownership, gets cards, deletes earlier cards for the selected document, inserts fresh card records, then returns cards. GET /api/flashcards/{document_id} returns saved cards after ownership check.

Viva question: Why delete old cards before saving new cards? Answer: The current design treats each generation as a fresh replacement set for that document.

15 Quiz Validation

Quiz generation needs stronger validation because it creates assessment data. For each AI item, backend checks question, correct_answer, explanation, source page and exactly four unique nonempty options. It also checks that correct_answer appears in options.

After validation, keep_distinct_study_items removes repetitive concepts. It uses study_words, set overlap and SequenceMatcher. focus_from_question creates a short label if provider did not provide one. This protects the frontend from repetitive or incomplete quiz content.

Validation rule	Why
Four options	Consistent multiple-choice layout.
Unique options	No duplicate answer buttons.
Correct answer in options	Quiz can be scored correctly.
Source page in range	Citation is meaningful.
Distinct concepts	Students receive useful practice.
Explanation present	Feedback after submission.

Viva question: Why not trust provider JSON directly? Answer: Provider output can miss fields, contain duplicates or give invalid source pages.

16 Local Quiz Fallback

ask_gemini_quiz catches provider 503 busy failures and returns None so the system can try local fallback instead of immediately displaying provider error. generate_quiz then calls make_local_quiz if provider questions are unavailable.

make_local_quiz uses source_chunks from SQLite. It chooses sentences with meaningful length, removes header-like or diagram-like fragments, removes near duplicates and creates question wording from focus. It selects three sentences from different topics as distractors. The source sentence becomes correct answer.

This fallback is intentionally source-grounded. It is not as natural as Gemini and it should not pretend to be. Its value is reliability: a student can continue learning when a network service is temporarily busy.

Viva question: What did the provider-busy fix change? Answer: Quiz now uses PDF text fallback instead of only returning a temporary busy error.

17 Quiz Submission and Progress

POST /api/quizzes/{quiz_id}/submit loads the quiz only if the quiz belongs to a document owned by the current user. It checks that each submitted answer is one of that question's options. It calculates score by comparing choice with stored answer.

The route inserts a quiz_attempts record using INSERT OR IGNORE so the first completed attempt is preserved. It returns score, total, XP earned and per-question correct answer, explanation and source page. GET /api/progress totals correct answers from attempts and computes XP as correct answers multiplied by ten.

```
score = sum(submission.answers[q["id"]] == q["answer"] for q in questions)
xp = correct_answers * 10
level = correct_answers // 10 + 1
```

Viva question: Why calculate answers on backend? Answer: Correct answers should remain trusted server data until submission.

18 Study Router Library and Subjects

create_router in study.py creates APIRouter with prefix /api/study. The inner owned function checks an item ID and user ID in a fixed allowed table name. Route implementations call it before editing, downloading or deleting user-owned items.

Library routes list subjects and documents, add a subject, assign a document to a subject, download original PDF and remove a document. Removal deletes related chunks, cards, quizzes and subject link, clears document reference from conversations, then deletes original file.

Route	Purpose
GET /api/study/library	Return user's subjects and documents.
POST /api/study/subjects	Create unique user subject.
PUT /api/study/documents/id/subject	Assign or clear subject.
GET /api/study/documents/id/file	Download owned original PDF.
DELETE /api/study/documents/id	Delete document and linked data.
GET /api/study/summary	Dashboard counts and recent items.

Viva question: Why delete linked chunks when document is deleted? Answer: They no longer have a source document and should not appear in retrieval.

19 Saved Conversations Revision and Tasks

Conversation routes list, load, save and delete saved study chats. Messages are stored as JSON. save_conversation uses an SQLite upsert: INSERT ... ON CONFLICT(id) DO UPDATE. It checks that an existing conversation is not owned by another user.

Revision routes store topic name, subject ID and a limited status such as learning, needs revision or confident. Task routes store title, kind, due date, status, subject, priority and notes. Every update and delete checks ownership before SQL executes.

Feature	CRUD routes
Conversations	GET list, GET one, PUT save, DELETE remove.
Revision topics	GET list, POST add, PATCH status, DELETE remove.
Tasks	GET list, POST add, PUT update, DELETE remove.

Viva question: What does CRUD mean? Answer: Create, Read, Update and Delete: the basic lifecycle of stored records.

20 Error Handling and Status Design

The backend uses HTTPException for expected failures. It chooses status codes to help frontend show good messages. A missing login is 401. A non-owned item returns 404 so the system does not reveal whether another user's record exists. A PDF too large is 413.

Status	Backend meaning	Frontend reaction
400	Bad file or input	Tell user what to correct.
401	Missing or bad token	Ask user to sign in.
404	Item unavailable or not owned	Show not found without leaking data.
409	Duplicate subject or account	Ask for different unique value.
413	PDF too large	Choose smaller PDF.
422	Cannot produce valid study item	Try clearer material.
502	Provider output incomplete	Try generation again.
503	Provider service busy	Use fallback or retry.

Do not expose raw stack traces or provider secrets to a student. chat_failure converts technical exceptions into useful messages. Logs may keep technical evidence for developer debugging.

Viva question: Why use 404 for not-owned record? Answer: It avoids revealing whether a private record exists for another user.

21 Backend Testing

Tests live in backend/tests. pytest uses isolated temporary environments and monkeypatching to avoid real provider calls. Tests cover API behaviour, chat fallback, profile, planner, streams, full PDF handling, reliability and workspace routes.

Test area	Example question tested
Authentication	Can signup and login work?
Upload	Are PDF validation and extraction rules followed?
Chat	Does fallback and streaming behave correctly?
Quiz reliability	Are duplicates filtered and busy fallback available?
Workspace	Are protected study routes owned by correct user?
Profile	Are preferences and avatar saved safely?

Testing does not mean the project has no bugs. It means important expected behaviour is checked repeatedly after code changes. A production frontend build also checks TypeScript and framework compilation.

Viva question: Why mock Gemini in tests? Answer: Tests should be repeatable, fast and not depend on internet quota or a real secret key.

22 Run Debug and Monitor Backend

```
cd backend
py -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

Open http://127.0.0.1:8000/health to check server. Open /docs to see FastAPI documentation and manually test routes after authenticating where required. The project launcher starts this backend alongside frontend.

- If port 8000 is already used, stop old service before restart.
- If API fails, inspect backend log before changing code.
- If PDF works but AI fails, check provider key and network; retrieval may still work.
- If tests fail, run the named test first and read error message.
- Never commit .env, database, uploads or virtual environment.

Viva question: What does --reload do? Answer: It restarts development server when Python files change; it is useful locally, not a production server setting.

23 Backend API Reference

Group	Main endpoints
Health	GET /health
Auth	POST signup, POST login, GET me
Profile	PUT preferences, PUT profile, PUT avatar
Documents	POST upload, GET documents
Chat	GET status, POST chat, POST chat/stream
Cards	POST generate, GET saved cards
Quiz	POST generate, POST submit
Progress	GET progress
Study router	Library, subjects, file download, conversations, topics, tasks, summary

This is a route map, not a claim that every URL is public. Most routes require Authorization Bearer token. The frontend uses a selected API base and includes that token in headers after login.

Viva question: What endpoint would the frontend call to generate a quiz? Answer: POST /api/quizzes/generate with document_id and count.

24 Backend Viva Questions

Viva question: What is the role of backend? Answer: Validate client requests, protect data, process PDFs, store records, retrieve relevant chunks and call AI providers.

Viva question: Why FastAPI? Answer: Clear Python routes, Pydantic validation, dependencies and automatic docs.

Viva question: Why SQLite? Answer: Simple local relational storage suitable for a BTech prototype.

Viva question: How is password safe? Answer: PBKDF2 HMAC SHA 256 with random salt and 120000 iterations; raw password is not stored.

Viva question: How does authorization work? Answer: current_user verifies token and owned_document or router owned checks user ID against resource.

Viva question: What is RAG? Answer: Retrieve relevant PDF chunks before generation.

Viva question: How does retrieval work? Answer: TF IDF and cosine similarity rank chunks against question.

Viva question: Why chunk PDF? Answer: Small page-linked pieces improve focused retrieval.

Viva question: What happens if Gemini fails? Answer: Backend can return source-text fallback and quiz uses local source-based fallback for busy provider.

Viva question: Why validate quiz JSON? Answer: Provider can return incomplete options or invalid answers.

Viva question: Why hide answers initially? Answer: Backend stores correct answer until student submits.

Viva question: How is score calculated? Answer: Compare submitted option to stored answer, then return score and explanation.

Viva question: What is main limitation? Answer: Text extraction from poor scans and lexical rather than semantic retrieval.

Viva question: Future backend upgrade? Answer: OCR, embeddings/vector database, PostgreSQL, standard sessions, monitoring and rate limiting.

25 Backend Code Walkthrough and Final Checklist

Open files in this exact order. First read requirements.txt so you know packages. Next read main.py top section: paths, db_connection and initialize_database. Then models, password functions and current_user. Then upload_document, split_text and retrieve. After that study chat provider functions, cards, quiz and progress. Finally read study.py route groups.

- I can explain why backend exists.
- I can explain FastAPI route and Pydantic model.
- I can explain SQLite tables and user ownership.
- I can explain password hash, token and current_user.
- I can explain upload validation, PyMuPDF and chunking.
- I can explain TF IDF and cosine similarity.
- I can explain Gemini prompt, streaming and fallback.
- I can explain distinct quiz validation and local fallback.
- I can explain study router CRUD endpoints.
- I can explain tests, logs and production limitations.

Final Backend Explanation

StudyForge backend is a FastAPI application that protects student accounts, stores study data in SQLite, extracts and chunks uploaded PDF text, retrieves relevant chunks through TF IDF and cosine similarity, calls AI providers when configured and provides fallback behaviour when they are unavailable. It exposes protected APIs for chat, flashcards, quizzes, profile, library, planner, revision and progress. Its key engineering priorities are validation, ownership, source grounding and useful failure handling.